

SKILL_WEEK - 11 (SESSION NUMBER - 21)

Task 1: To connect modules, validate interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components.

```
#include <stdio.h>
#include <string.h>

int parser_module(const char *input)
{
    if (input == NULL || strlen(input) == 0)
    {
        printf("Parser Error: Empty command.\n");
        return 0;
    }

    printf("Parser Module: Command parsed successfully.\n");
    return 1;
}

int executor_module(const char *input)
{
    if (strcmp(input, "test") == 0)
    {
        printf("Executor Module: Command executed successfully.\n");
        return 1;
    }

    printf("Executor Error: Unsupported command.\n");
    return 0;
}

int main()
{
    char command[100];

    printf("Enter command: ");
    scanf("%99s", command);

    printf("\nConnecting modules...\n");

    if (!parser_module(command))
    {
        printf("Execution stopped safely.\n");
        return 1;
    }

    if (!executor_module(command))
    {
        printf("Error handled gracefully.\n");
        return 1;
    }

    printf("End-to-end execution completed successfully.\n");
    printf("Module interfaces validated successfully.\n");

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session21_task1.c -o bin/session21_task1
./bin/session21_task1
Enter command: test

Connecting modules...
Parser Module: Command parsed successfully.
Executor Module: Command executed successfully.
End-to-end execution completed successfully.
Module interfaces validated successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To detect runtime errors, generate messages, log failures, handle invalid syntax, recover gracefully, and test error scenarios.

```
        return 0;
    }

    if (strcmp(command, "run") != 0)
    {
        printf("Syntax Error: Invalid command.\n");

        logfile = fopen("error_log.txt", "a");

        if (logfile != NULL)
        {
            fprintf(logfile,
                    "Error: Invalid command -> %s\n",
                    command);
            fclose(logfile);
        }

        printf("Failure logged successfully.\n");
        printf("Program recovered gracefully.\n");

        return 0;
    }

    printf("Command executed successfully.\n");

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session21_task2.c -o bin/session21_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session21_task2
Enter command: hello
Syntax Error: Invalid command.
Failure logged successfully.
Program recovered gracefully.
amrutha@Amruthas-MacBook-Air OSSP % cat error_log.txt
Error: Invalid command -> hello
amrutha@Amruthas-MacBook-Air OSSP %
```

(SESSION NUMBER - 22)

Task 1: To run Valgrind, identify leaks, fix memory errors, release resources, verify cleanup, and document findings.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *data;

    data = (char *)malloc(100 * sizeof(char));

    if (data == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    strcpy(data, "OSSP Memory Management Test");

    printf("Allocated Data: %s\n", data);

    free(data);
    data = NULL;

    printf("Memory released successfully.\n");
    printf("Cleanup completed without memory leaks.\n");

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air: OSSP % /usr/bin/leaks --atExit -- ./bin/session22_task1
session22_task1(21018) MallocStackLogging: could not tag MSL-related memory as no footprint, so those pages will be included in process footprint - No such file or directory (2)
session22_task1(21018) MallocStackLogging: recording malloc (and VM allocation) stacks using lite mode
Allocated Data: OSSP Memory Management Test
Memory released successfully.
Cleanup completed without memory leaks.
Process 21018 is not debuggable. Due to security restrictions, leaks can only show or save contents of readonly memory of restricted processes.

Process:      session22_task1 [21018]
Path:         /Users/USER/*/session22_task1
Load Address: 0x10406c000
Identifier:   session22_task1
Version:      0
Code Type:   ARM64
Platform:    macOS
Parent Process: leaks [21017]
Target Type:  live task

Date/Time:    2026-09-29 19:46:19.397 +0530
Launch Time:  2026-09-29 19:46:19.363 +0530
OS Version:   macOS 26.6.2 (25G83)
Report Version: 7
Analysis Tool: /usr/bin/leaks

Physical footprint:      2032K
Physical footprint (peak): 2032K
Idle exit:               untracked
-----

leaks Report Version: 4.0, multi-line stacks
Process 21018: 190 nodes malloced for 16 KB
Process 21018: 0 leaks for 0 total leaked bytes.
```

Task 2: To design test cases, create test scripts, automate execution, verify outputs, compare results, and generate reports.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char input[50];

    printf("Enter command: ");
    scanf("%49s", input);

    if (strcmp(input, "hello") == 0)
    {
        printf("Output: Hello OSSP\n");
    }
    else if (strcmp(input, "test") == 0)
    {
        printf("Output: Test Successful\n");
    }
    else
    {
        printf("Output: Invalid Input\n");
    }

    return 0;
}
```

```
#!/bin/bash

echo "Running Automated Test Cases"
echo "-----"

echo "hello" | ./bin/session22_task2 > output1.txt
echo "test" | ./bin/session22_task2 > output2.txt
echo "abc" | ./bin/session22_task2 > output3.txt

echo "Test Case 1 Output:"
cat output1.txt

echo
echo "Test Case 2 Output:"
cat output2.txt

echo
echo "Test Case 3 Output:"
cat output3.txt

echo
echo "Automated testing completed."
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % nano src/session22_task2.c
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/session22_task2.c -o bin/session22_task2
[amrutha@Amruthas-MacBook-Air OSSP % nano test_session22.sh
[amrutha@Amruthas-MacBook-Air OSSP % chmod +x test_session22.sh
[amrutha@Amruthas-MacBook-Air OSSP % ./test_session22.sh
Running Automated Test Cases
-----
Test Case 1 Output:
Enter command: Output: Hello OSSP

Test Case 2 Output:
Enter command: Output: Test Successful

Test Case 3 Output:
Enter command: Output: Invalid Input

Automated testing completed.
amrutha@Amruthas-MacBook-Air OSSP %
```